```
from google.colab import drive
import os, glob, subprocess, glob, numpy as np, matplotlib.pyplot as
plt
from scipy import signal
import soundfile as sf
from sklearn.decomposition import NMF

from google.colab import drive
import os
drive.mount('/content/drive')
sound_dir = '/content/drive/MyDrive/Sound/60s'
os.chdir(sound_dir)
print(f"Current working directory: {os.getcwd()}")
```

```
Mounted at /content/drive
Current working directory: /content/drive/MyDrive/Sound/60s
```

```
m4a_files = glob.glob('*.m4a')
print("Found .m4a files:", m4a_files)
```

```
Found .m4a files: ['60s-100.m4a', '60s-90.m4a', '60s-80.m4a', '60s-
70.m4a', '60s-60.m4a', '60s-50-x.m4a', '60s-50.m4a', '60s-40.m4a',
'60s-30.m4a', '60s-20.m4a', '60s-10.m4a']
```

```
def convert_m4a_to_wav(m4a_file, wav_file):
  res = subprocess.run(['ffmpeg', '-y', '-i', m4a_file, wav_file],
capture_output=True, text=True)
  print("ffmpeg returncode:", res.returncode)
  if res.returncode != 0:
      print("ffmpeg stderr:", res.stderr[:1000])
      raise RuntimeError("ffmpeg failed to convert the file")
  print("Converted file exists:", os.path.exists(wav_file))
  return res

def read_wav(wav_file):
  y, sr = sf.read(wav_file)
  if y.ndim > 1:
      y = np.mean(y, axis=1)
  y = y.astype(np.float32)
  duration = len(y) / sr
  return y, sr, duration

def STFT_shape(y, sr):  # Short-Time Fourier Transform
  n_fft = 2048
  hop_length = 512
  #
  f, t, Zxx = signal.stft(y, fs=sr, nperseg=n_fft, noverlap=n_fft-
hop_length, boundary=None)
  S = np.abs(Zxx)
  return S, f, t, Zxx, n_fft, hop_length
```

```
def separate_with_NMF(S, n_max_components): # Non-negative Matrix
Factorization (NMF) for separation
  S_max = S.max() if S.max() > 0 else 1.0
  S_norm = S / S_max + 1e-10
  recons_scores = {}
  for n_comp in range(2,n_max_components):
      model = NMF(n_components=n_comp, init='nndsvda', solver='mu',
beta_loss='kullback-leibler', max_iter=500, random_state=0)
      W = model.fit_transform(S_norm)
      H = model.components_
      S_approx = np.dot(W,H)
      err = np.linalg.norm(S_norm - S_approx, ord='fro')
      fit = 1 - (err / np.linalg.norm(S_norm, ord='fro'))
      recons_scores[n_comp] = fit
      print(f"n={n_comp} fit={fit:.4f}")
  return recons_scores, S_max, S_norm

def choose_best_n(recons_scores):
  prev = None
  best_n = 6
  for n, fit in sorted(recons_scores.items()):
      if prev is not None:
          if fit - prev < 0.02:
              best_n = n
              break
      prev = fit
  return best_n

def NMF_recompute(S, best_n):
  model = NMF(n_components=best_n, init='nndsvda', solver='mu',
beta_loss='kullback-leibler', max_iter=500, random_state=0)
  W = model.fit_transform(S_norm); H = model.components_
  components_S = []
  for k in range(best_n):
      comp = np.outer(W[:,k], H[k,:]) * S_max
      components_S.append(comp)
  components_S = np.array(components_S)
  return components_S

def mask_reconstruct(components_S, Zxx):
  eps = 1e-10
  sum_S = np.sum(components_S, axis=0) + eps
  masks = components_S / sum_S
  reconstructed = []
  out_files = []
  for k in range(best_n):
      masked = masks[k] * Zxx
      _, y_comp = signal.istft(masked, fs=sr, nperseg=n_fft,
noverlap=n_fft-hop_length, input_onesided=True)
      y_comp = y_comp[:len(y)]
```

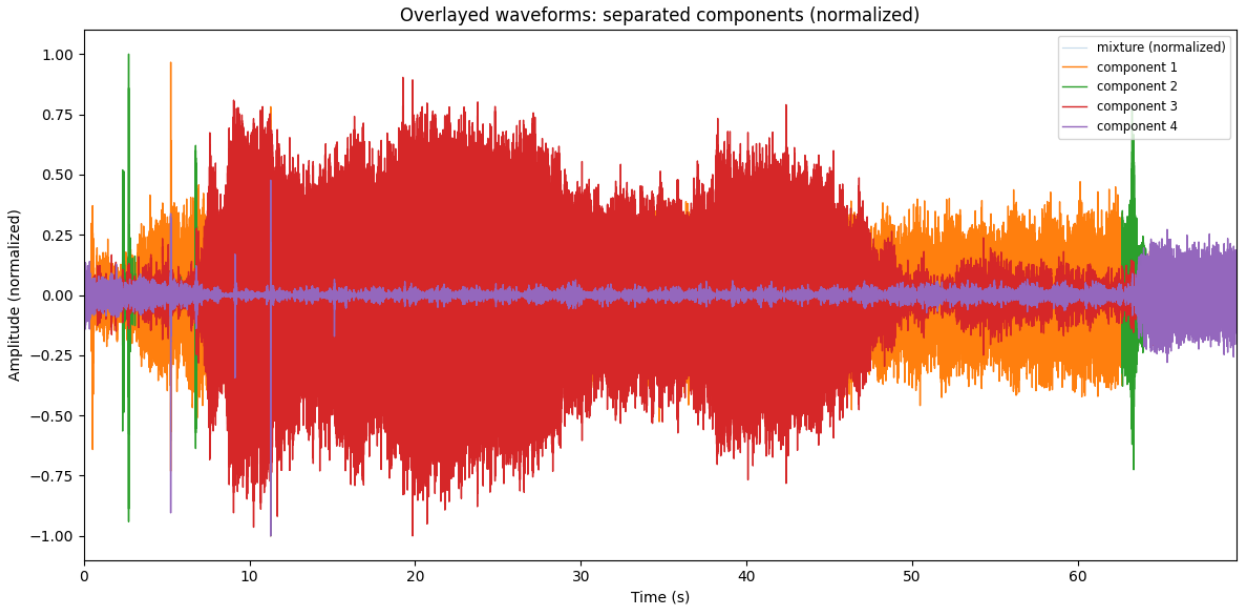
```
        reconstructed.append(y_comp)
        base, _ = os.path.splitext(m4a_file)
        outp = f"{base}_{k+1}.wav"
        sf.write(outp, y_comp, sr)
        out_files.append(outp)
    return masks, reconstructed, out_files

def plot_overlayed_waveforms(y, reconstructed, sr, duration):
    plt.figure(figsize=(12,6))
    t_axis = np.arange(len(y)) / sr
    plt.plot(t_axis, y / (np.max(np.abs(y)) + 1e-9), alpha=0.25,
linewidth=0.8, label='mixture (normalized)')
    for k, sig in enumerate(reconstructed):
        if np.max(np.abs(sig)) > 0:
            sign = sig / np.max(np.abs(sig))
        else:
            sign = sig
        # Create t_axis based on the length of the current signal
        t_axis_comp = np.arange(len(sign)) / sr
        plt.plot(t_axis_comp, sign, linewidth=1, label=f'component
{k+1}')
    plt.xlabel('Time (s)'); plt.ylabel('Amplitude (normalized)')
    plt.title('Overlayed waveforms: separated components (normalized)')
    plt.legend(loc='upper right', fontsize='small')
    plt.xlim(0, duration)
    plt.tight_layout()
    plt.show()
    plt.close()

def plot_individual_waveforms(y, reconstructed, sr, duration):
    n_signals = len(reconstructed)
    fig, axes = plt.subplots(n_signals + 1, 1, figsize=(12,
2*(n_signals+1)), sharex=True)

    # Plot the mixture on the first subplot
    t_axis = np.arange(len(y)) / sr
    axes[0].plot(t_axis, y / (np.max(np.abs(y)) + 1e-9), alpha=0.25,
linewidth=0.8, label='mixture (normalized)')
    axes[0].set_ylabel('Mixture')
    axes[0].legend(loc='upper right', fontsize='small')
    axes[0].set_title('Individual waveforms: separated components
(normalized)')

    # Plot each reconstructed component in its own subplot
    for k, sig in enumerate(reconstructed):
        if np.max(np.abs(sig)) > 0:
            sig_norm = sig / np.max(np.abs(sig))
        else:
            sig_norm = sig
        t_axis_comp = np.arange(len(sig_norm)) / sr
```

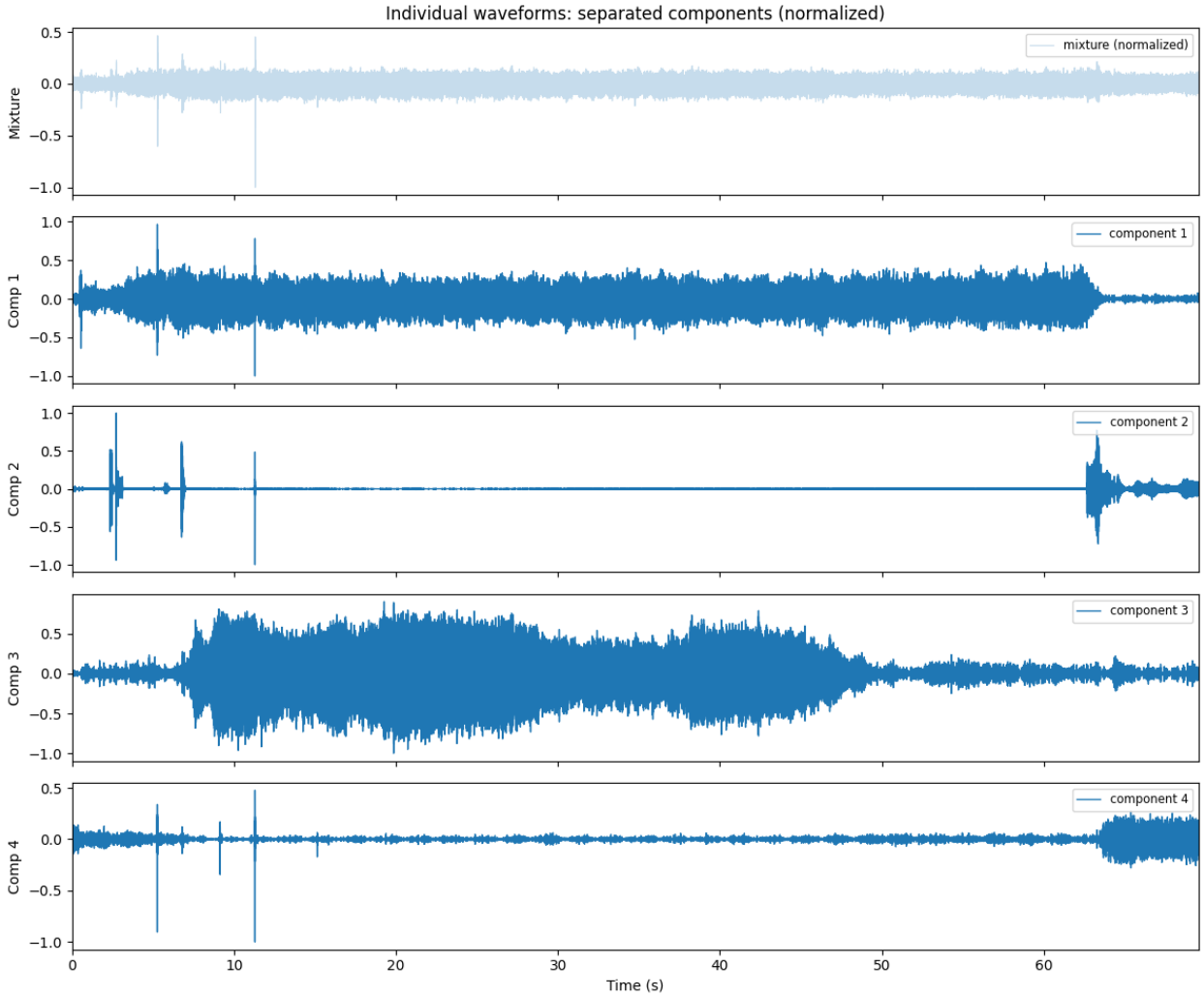
```
        axes[k+1].plot(t_axis_comp, sig_norm, linewidth=1,
label=f'component {k+1}')
        axes[k+1].set_ylabel(f'Comp {k+1}')
        axes[k+1].legend(loc='upper right', fontsize='small')

    axes[-1].set_xlabel('Time (s)')
    plt.xlim(0, duration)
    plt.tight_layout()
    plt.show()
    plt.close()
```

```
wav_files = []
for m4a_file in m4a_files:
    wav_file = os.path.splitext(m4a_file)[0] + '.wav'
    print(f"Converting {m4a_file} to {wav_file}...")
    res = convert_m4a_to_wav(m4a_file, wav_file)
    wav_files.append(wav_file)
    y, sr, duration = read_wav(wav_file)
    print(f"Loaded '{wav_file}' — duration: {duration:.2f}s, sample
rate: {sr} Hz, samples: {len(y)}")
    S, f, t, Zxx, n_fft, hop_length = STFT_shape(y, sr)
    print("STFT shape:", S.shape)
    n_max_components = 7
    recons_scores, S_max, S_norm = separate_with_NMF(S,
n_max_components)
    best_n = choose_best_n(recons_scores)
    print("Chosen components:", best_n)
    components_S = NMF_recompute(S, best_n)
    masks, reconstructed, out_files = mask_reconstruct(components_S,
Zxx)
    print("Saved outputs:", out_files)
    plot_overlayed_waveforms(y, reconstructed, sr, duration)
    plot_individual_waveforms(y, reconstructed, sr, duration)
print("Converted .wav files:", wav_files)
```

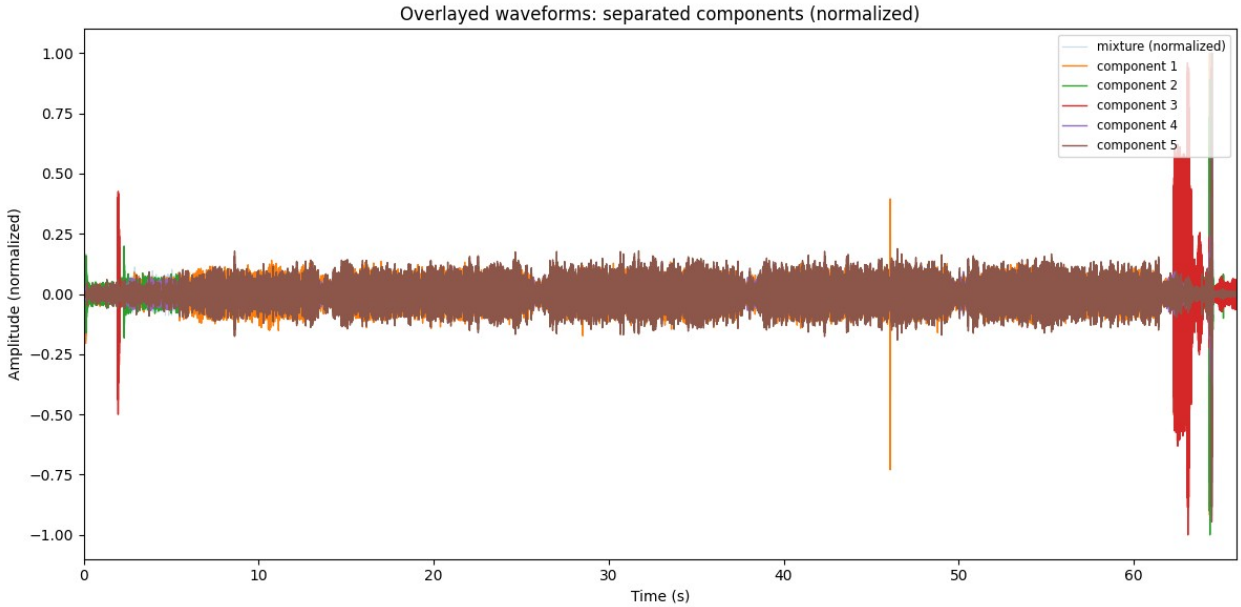
```
Converting 60s-100.m4a to 60s-100.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-100.wav' — duration: 69.57s, sample rate: 48000 Hz,
samples: 3339264
STFT shape: (1025, 6519)
n=2 fit=0.6335
n=3 fit=0.6579
n=4 fit=0.6704
n=5 fit=0.6999
n=6 fit=0.7215
Chosen components: 4
```

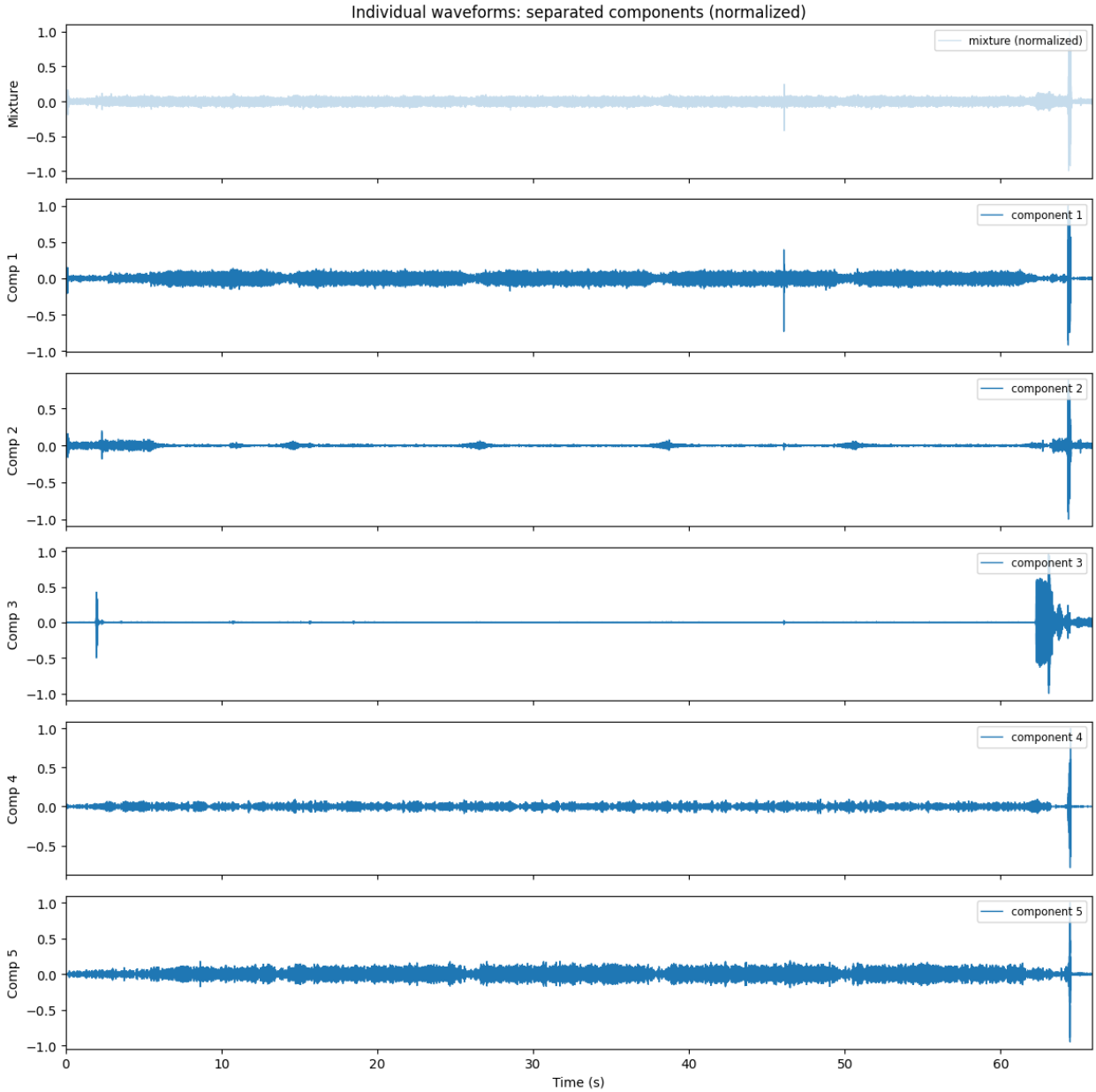
```
Saved outputs: ['60s-100_1.wav', '60s-100_2.wav', '60s-100_3.wav',
'60s-100_4.wav']
```





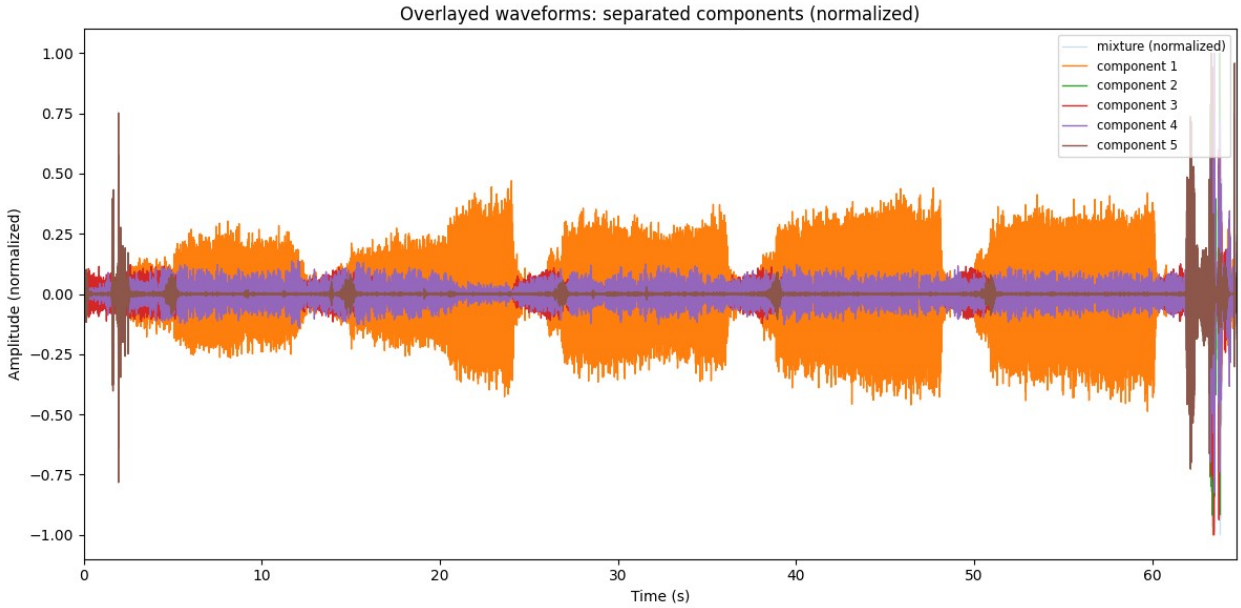

```
Converting 60s-90.m4a to 60s-90.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-90.wav' — duration: 65.88s, sample rate: 48000 Hz,
samples: 3162112
STFT shape: (1025, 6173)
n=2 fit=0.5233
n=3 fit=0.5697
n=4 fit=0.5961
n=5 fit=0.6093
n=6 fit=0.6454
Chosen components: 5
Saved outputs: ['60s-90_1.wav', '60s-90_2.wav', '60s-90_3.wav', '60s-
90_4.wav', '60s-90_5.wav']
```

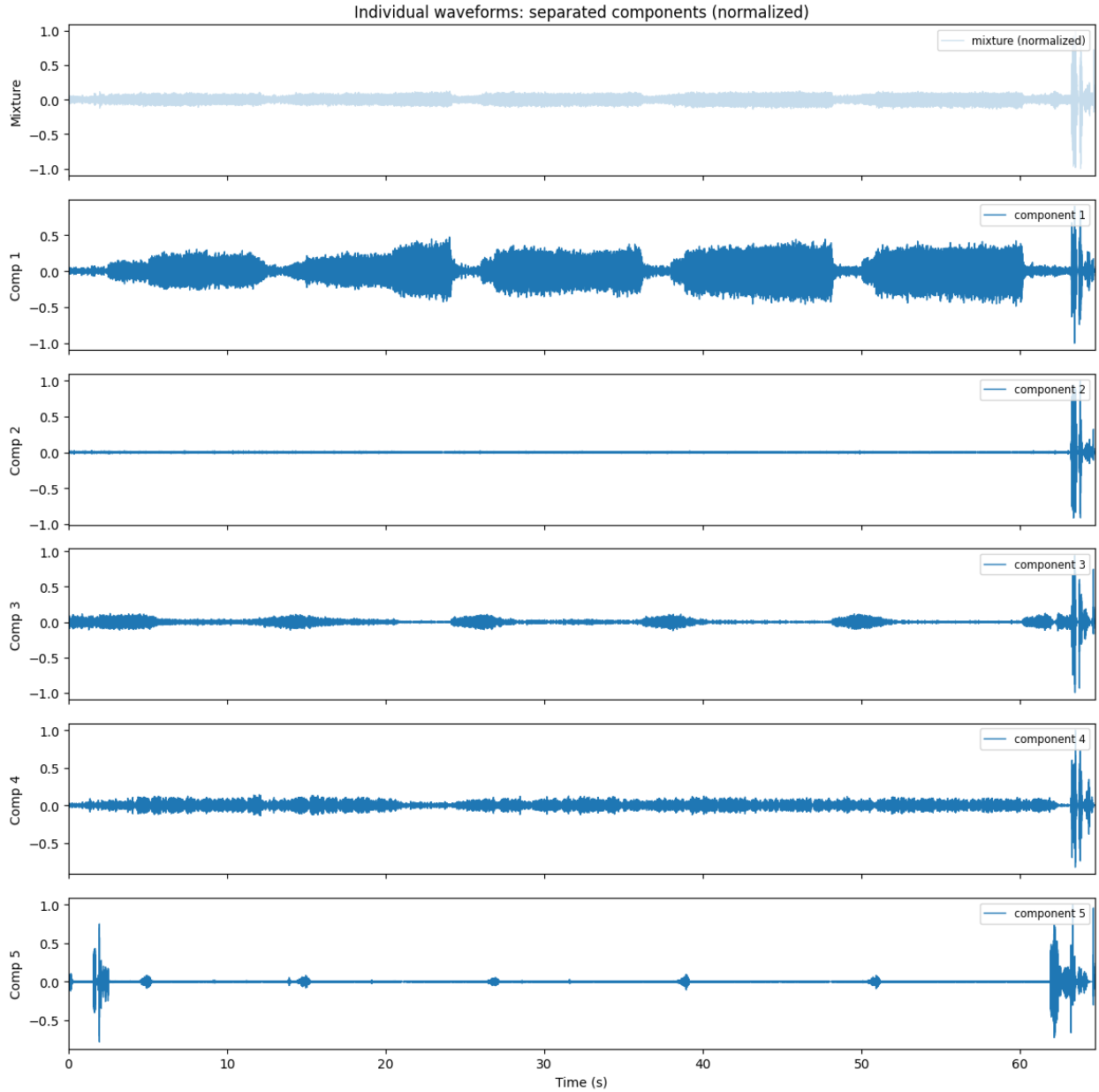

Overlayed waveforms: separated components (normalized)
mixture (normalized)
component 1
component 2
component 3
component 4
component 5
Amplitude (normalized)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
10
20
30
40
50
60
Time (s)




```
Converting 60s-80.m4a to 60s-80.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-80.wav' — duration: 64.73s, sample rate: 48000 Hz,
samples: 3106816
STFT shape: (1025, 6065)
n=2 fit=0.5475
n=3 fit=0.6167
n=4 fit=0.6481
n=5 fit=0.6667
```

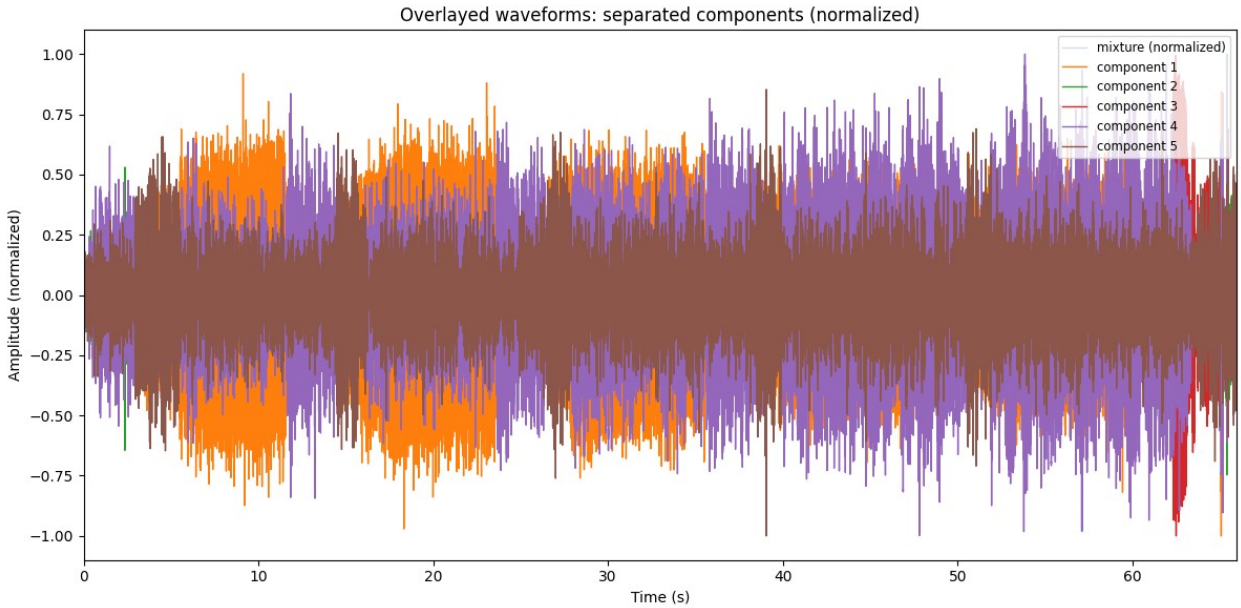
```
/usr/local/lib/python3.12/dist-packages/sklearn/decomposition/
_nmf.py:1742: ConvergenceWarning: Maximum number of iterations 500
reached. Increase it to improve convergence.
  warnings.warn(

n=6 fit=0.7160
Chosen components: 5
Saved outputs: ['60s-80_1.wav', '60s-80_2.wav', '60s-80_3.wav', '60s-
80_4.wav', '60s-80_5.wav']
```

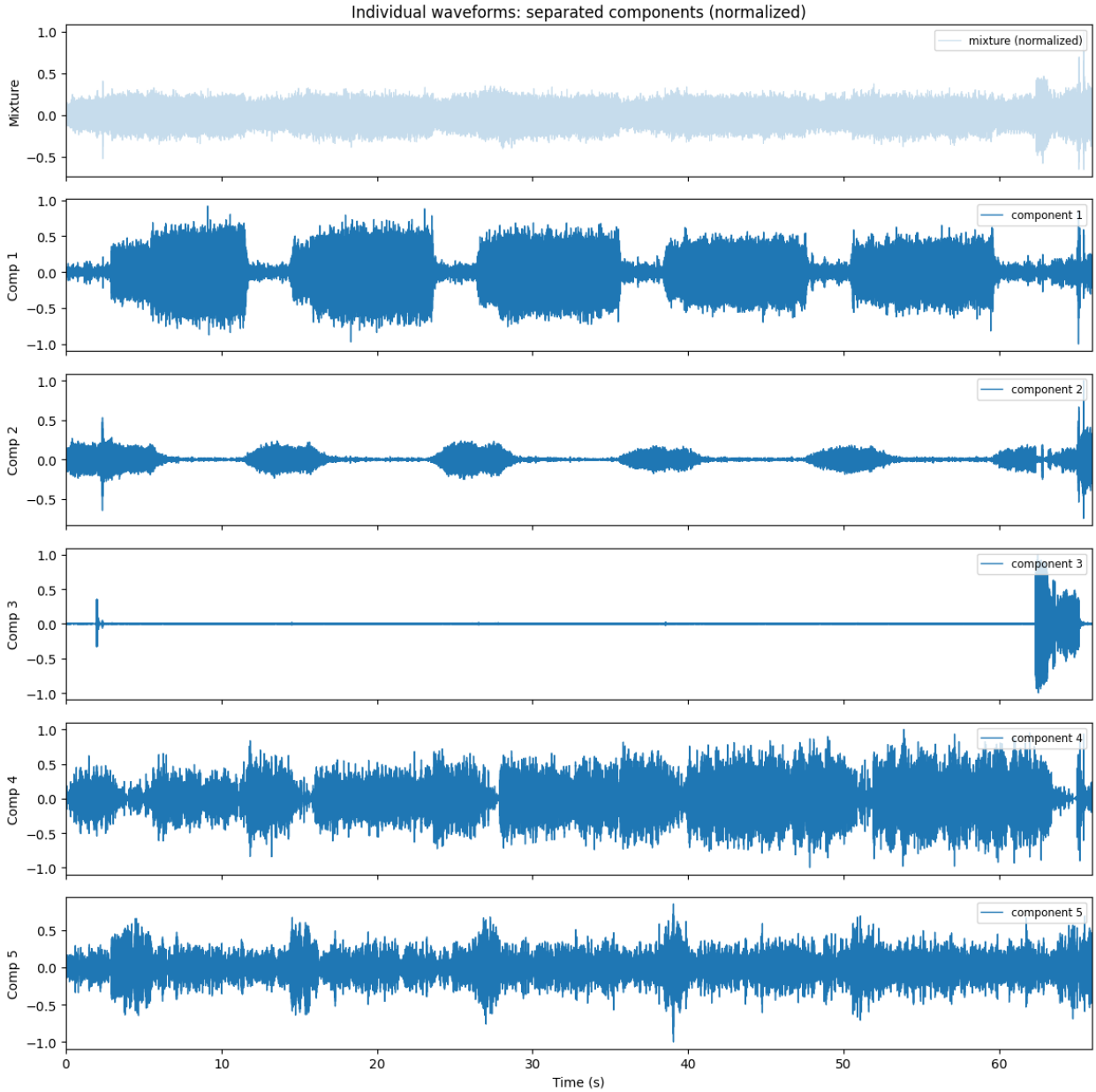





```
Converting 60s-70.m4a to 60s-70.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-70.wav' — duration: 65.96s, sample rate: 48000 Hz,
samples: 3166208
STFT shape: (1025, 6181)
n=2 fit=0.5711
n=3 fit=0.6301
n=4 fit=0.6733
n=5 fit=0.6871
n=6 fit=0.7003
Chosen components: 5
```

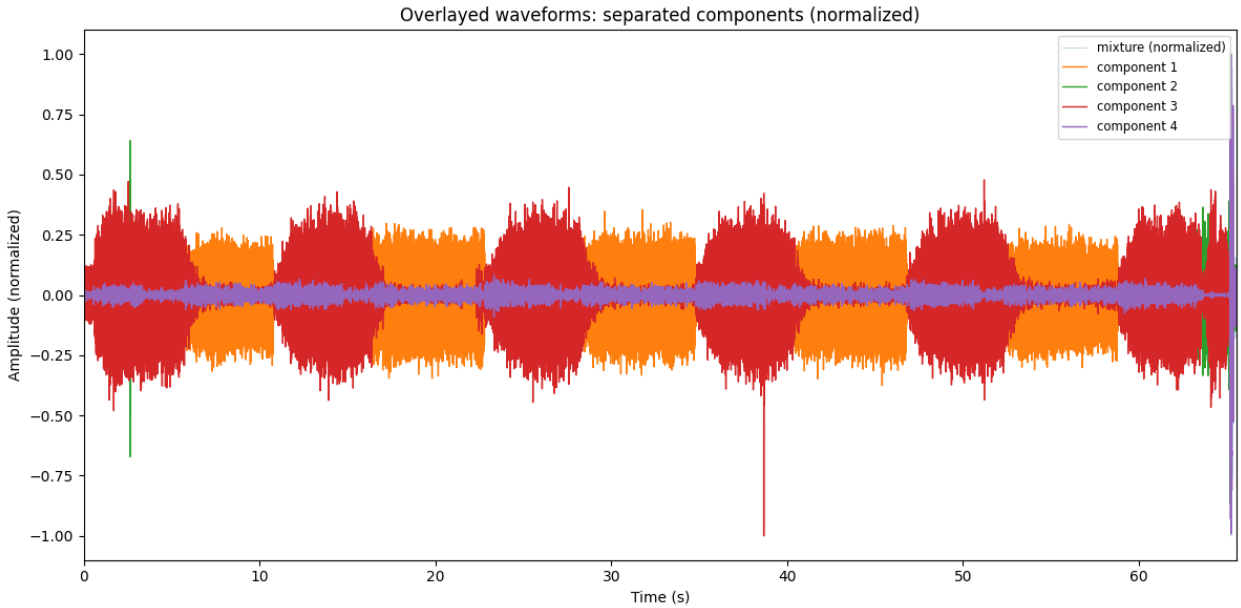
```
Saved outputs: ['60s-70_1.wav', '60s-70_2.wav', '60s-70_3.wav', '60s-70_4.wav', '60s-70_5.wav']
```

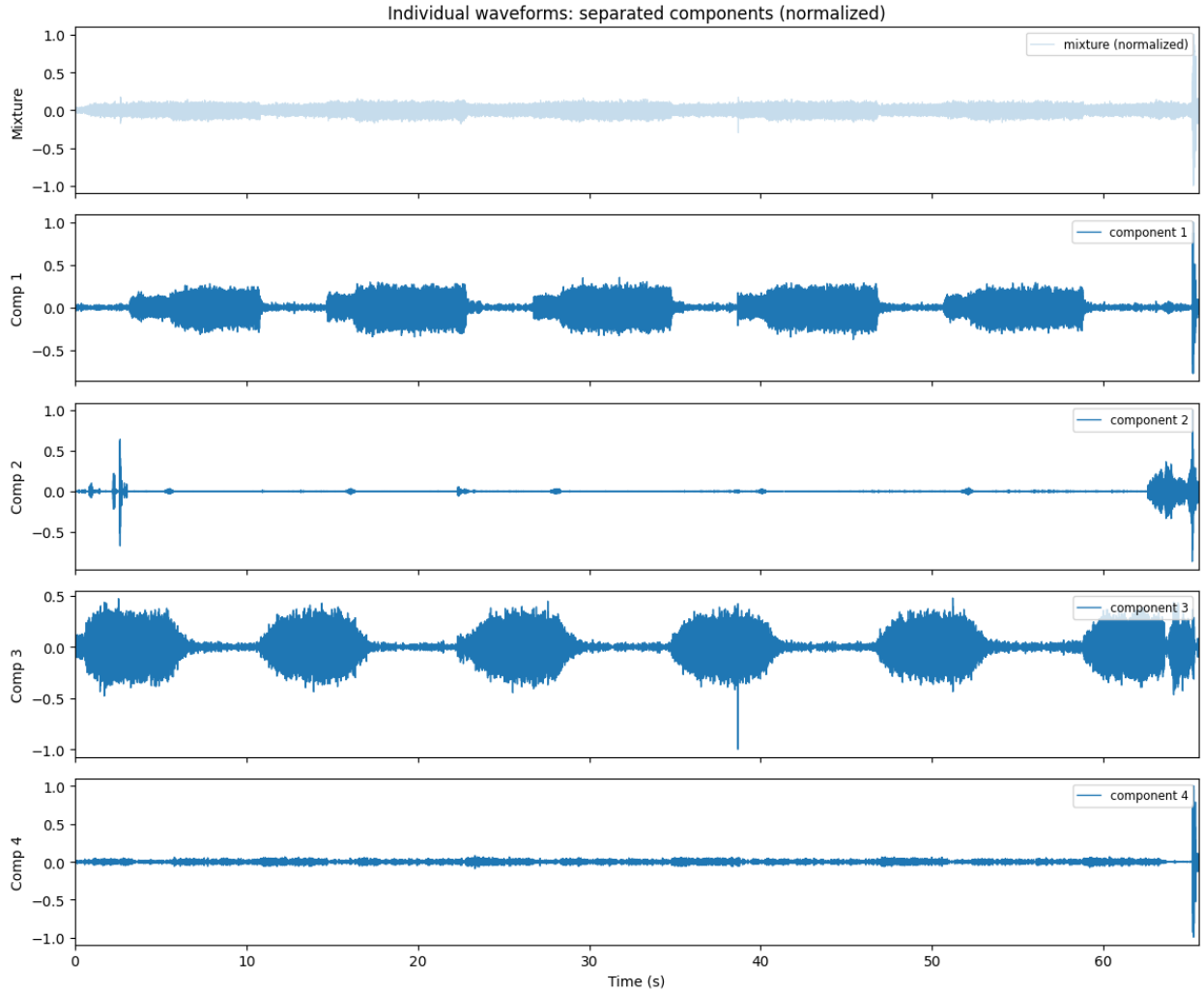




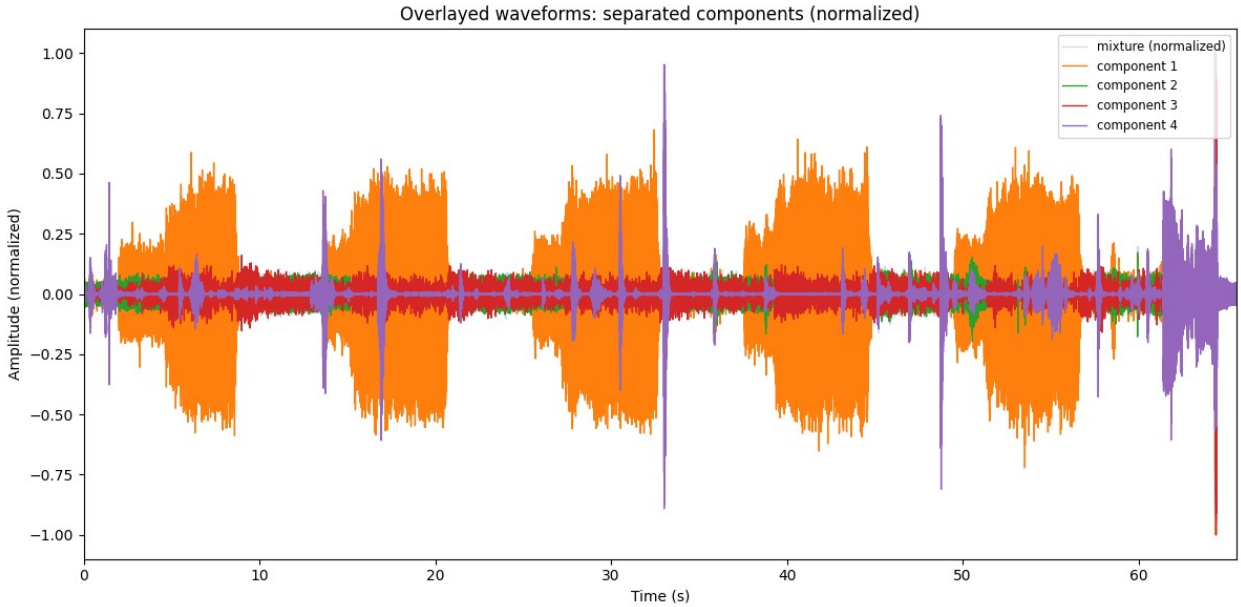

```
Converting 60s-60.m4a to 60s-60.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-60.wav' — duration: 65.56s, sample rate: 48000 Hz,
samples: 3146752
STFT shape: (1025, 6143)
n=2 fit=0.5284
n=3 fit=0.5966
n=4 fit=0.6148
n=5 fit=0.6218
n=6 fit=0.6318
Chosen components: 4
```

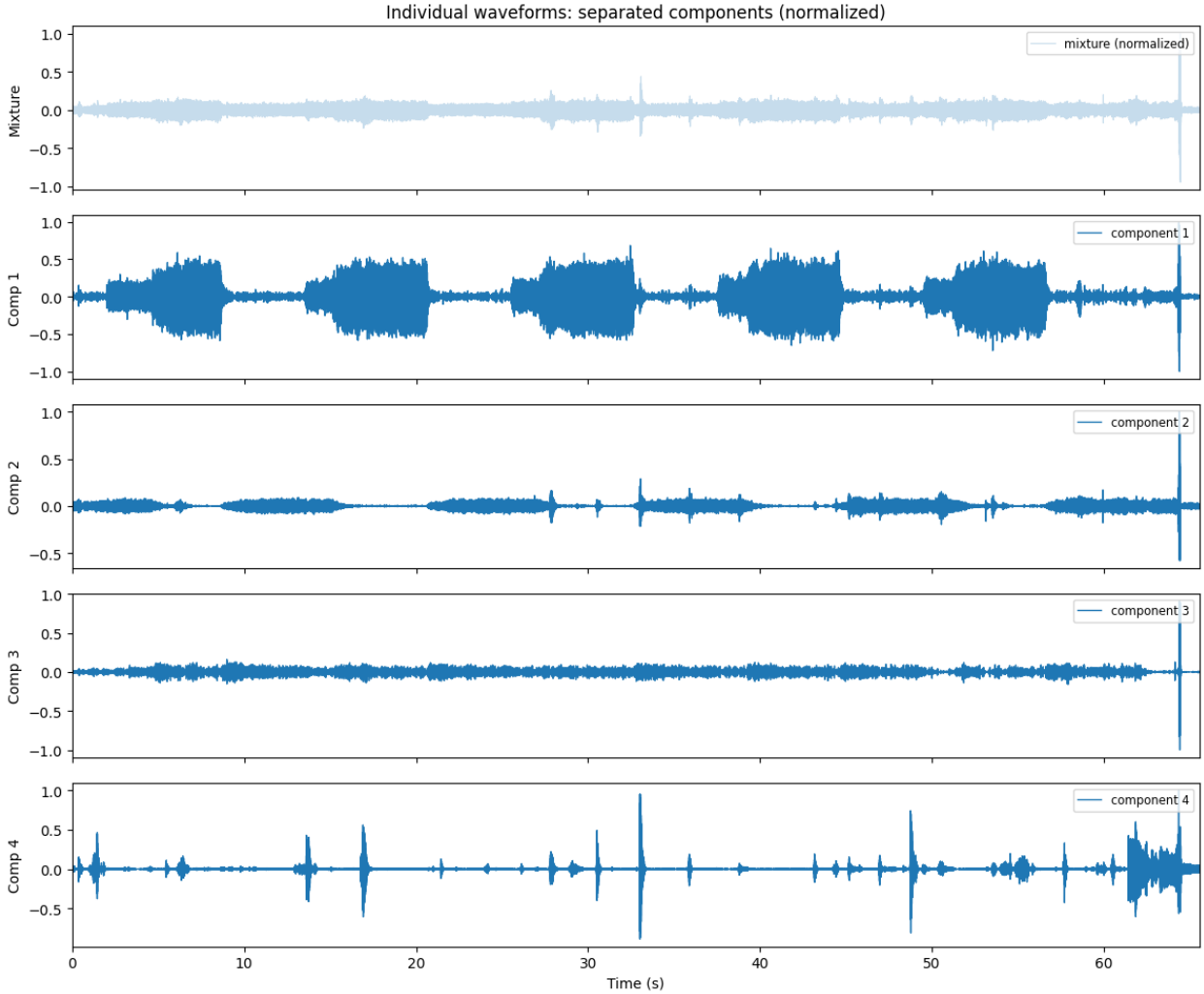
```
Saved outputs: ['60s-60_1.wav', '60s-60_2.wav', '60s-60_3.wav', '60s-60_4.wav']
```



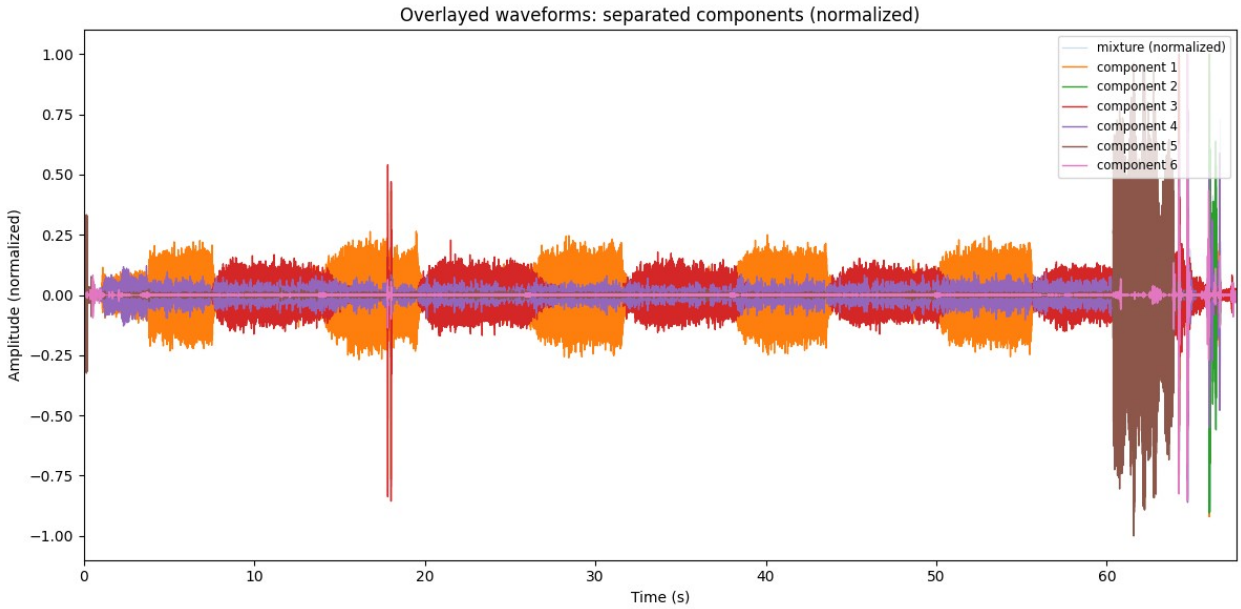


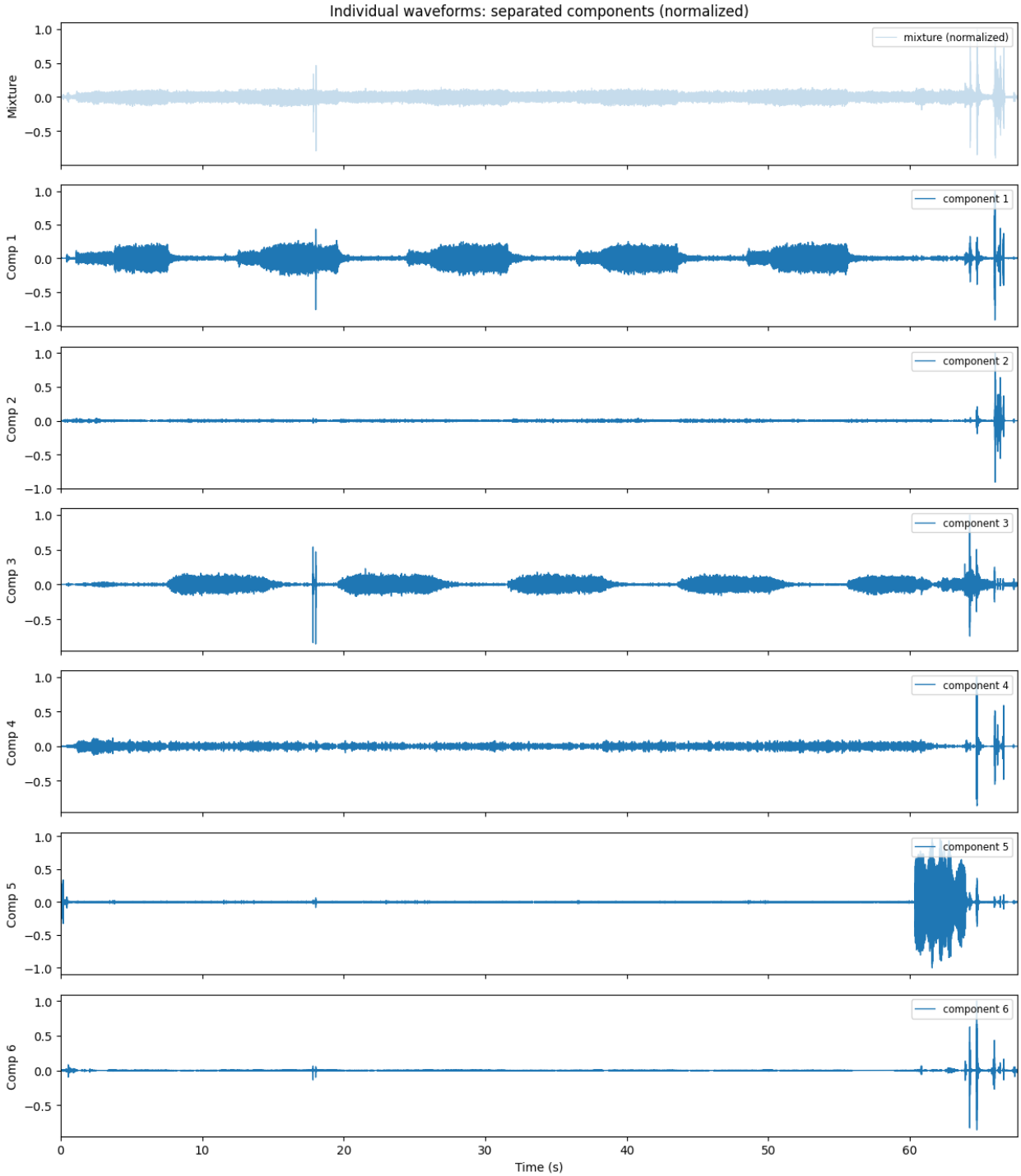

```
Converting 60s-50-x.m4a to 60s-50-x.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-50-x.wav' — duration: 65.58s, sample rate: 48000 Hz,
samples: 3147776
STFT shape: (1025, 6145)
n=2 fit=0.5427
n=3 fit=0.5942
n=4 fit=0.6077
n=5 fit=0.6372
n=6 fit=0.6529
Chosen components: 4
Saved outputs: ['60s-50-x_1.wav', '60s-50-x_2.wav', '60s-50-x_3.wav',
'60s-50-x_4.wav']
```


Overlayed waveforms: separated components (normalized)
mixture (normalized)
component 1
component 2
component 3
component 4
Amplitude (normalized)
Time (s)


Individual waveforms: separated components (normalized)
Mixture
mixture (normalized)
Comp 1
component 1
Comp 2
component 2
Comp 3
component 3
Comp 4
component 4
Time (s)

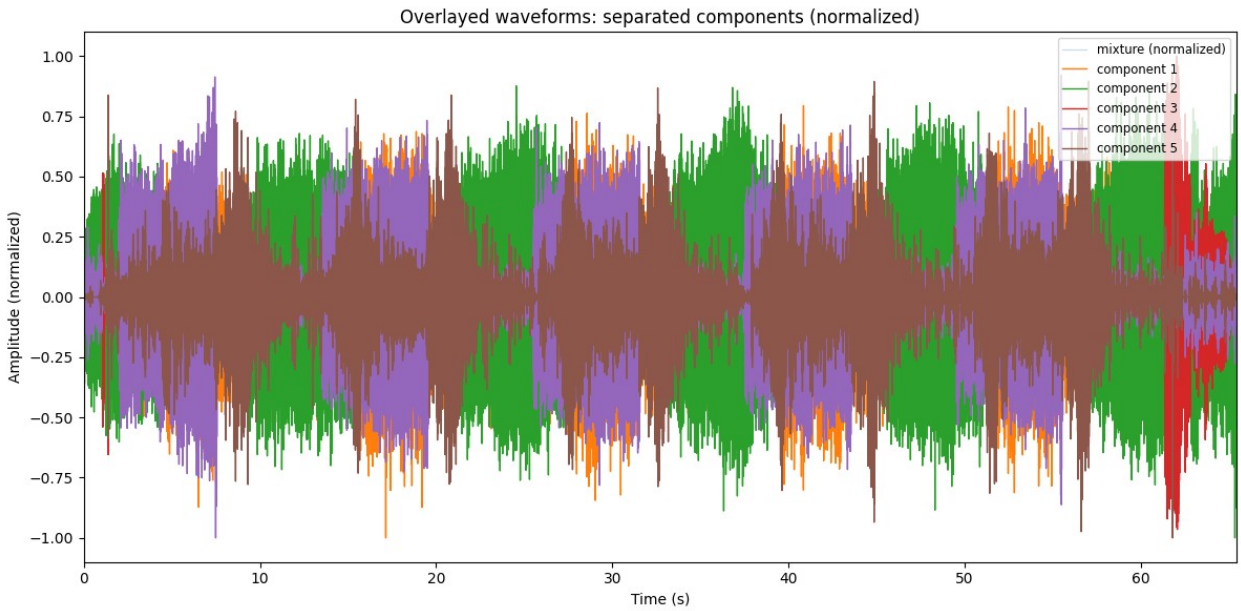
```
Converting 60s-50.m4a to 60s-50.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-50.wav' — duration: 67.61s, sample rate: 48000 Hz,
samples: 3245056
STFT shape: (1025, 6335)
n=2 fit=0.4290
n=3 fit=0.4827
n=4 fit=0.5441
n=5 fit=0.5967
n=6 fit=0.6578
Chosen components: 6
Saved outputs: ['60s-50_1.wav', '60s-50_2.wav', '60s-50_3.wav', '60s-
50_4.wav', '60s-50_5.wav', '60s-50_6.wav']
```

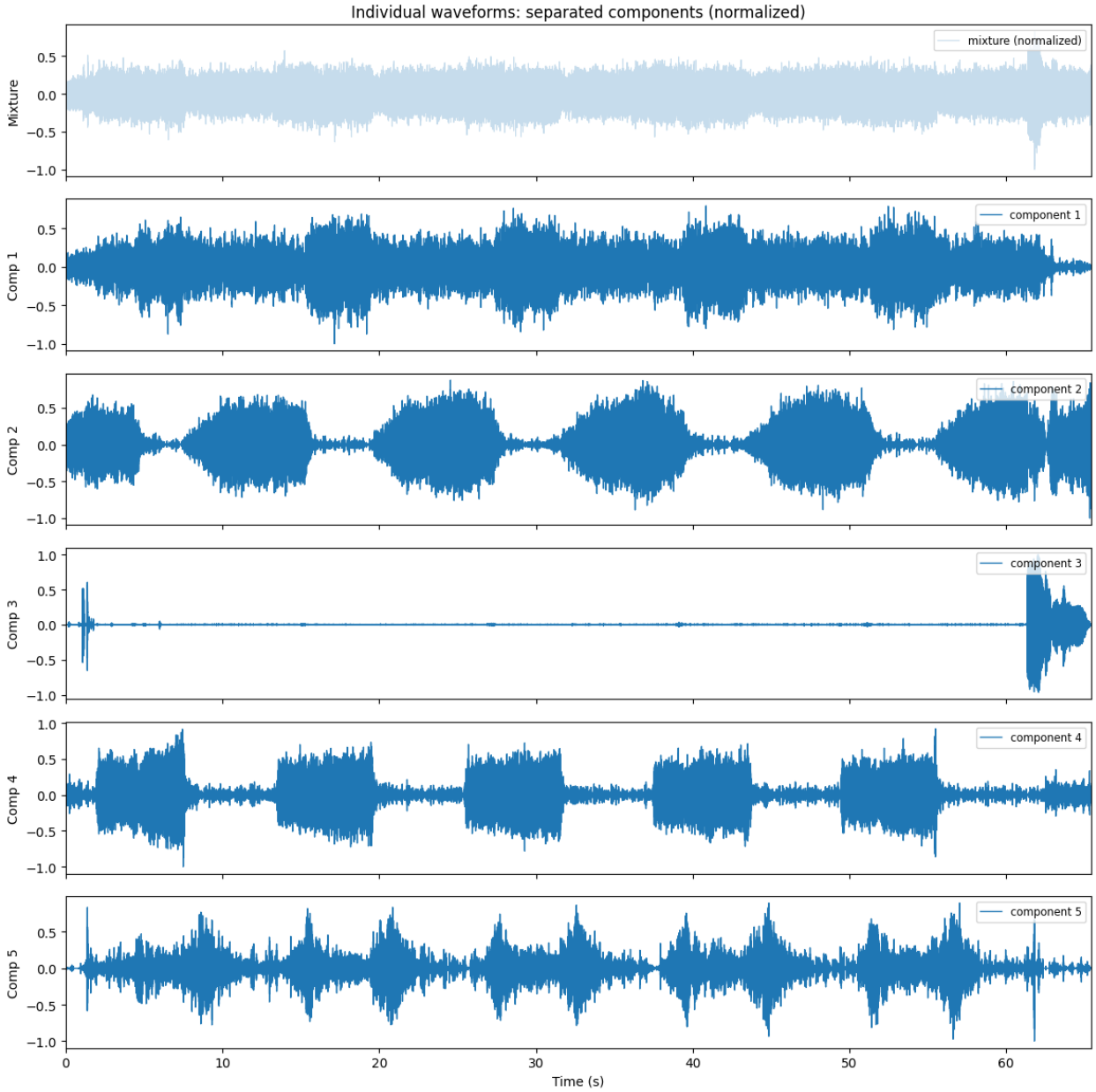





```
Converting 60s-40.m4a to 60s-40.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-40.wav' — duration: 65.43s, sample rate: 48000 Hz,
samples: 3140608
STFT shape: (1025, 6131)
```

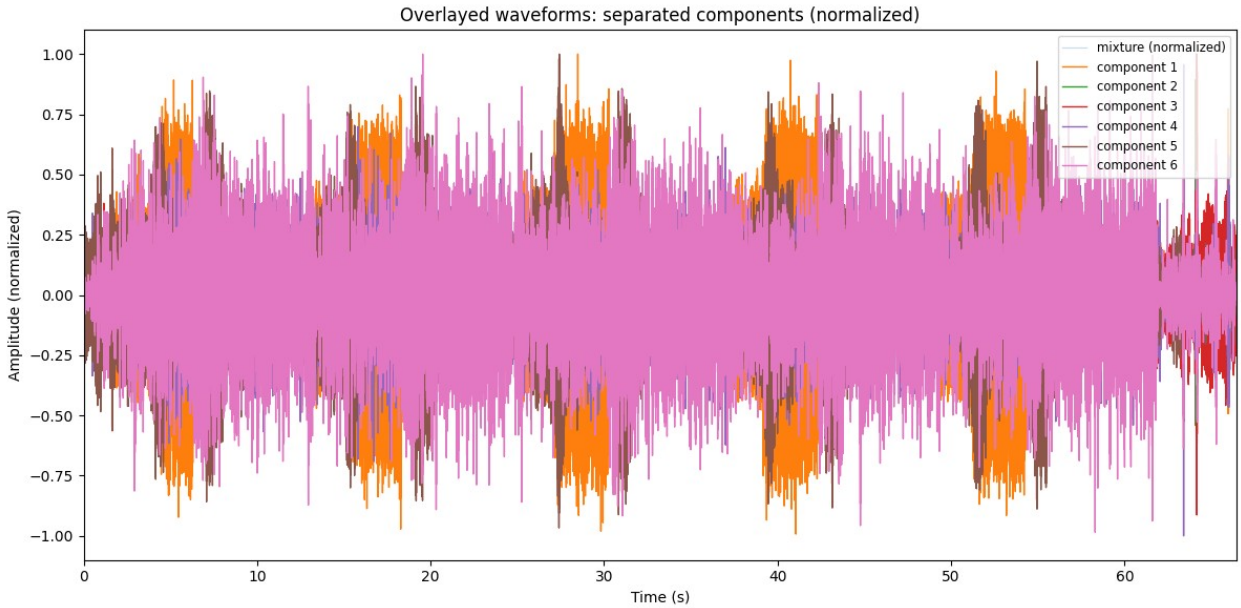
```
n=2 fit=0.5565
n=3 fit=0.6132
n=4 fit=0.6487
n=5 fit=0.6658
n=6 fit=0.6783
Chosen components: 5
Saved outputs: ['60s-40_1.wav', '60s-40_2.wav', '60s-40_3.wav', '60s-40_4.wav', '60s-40_5.wav']
```

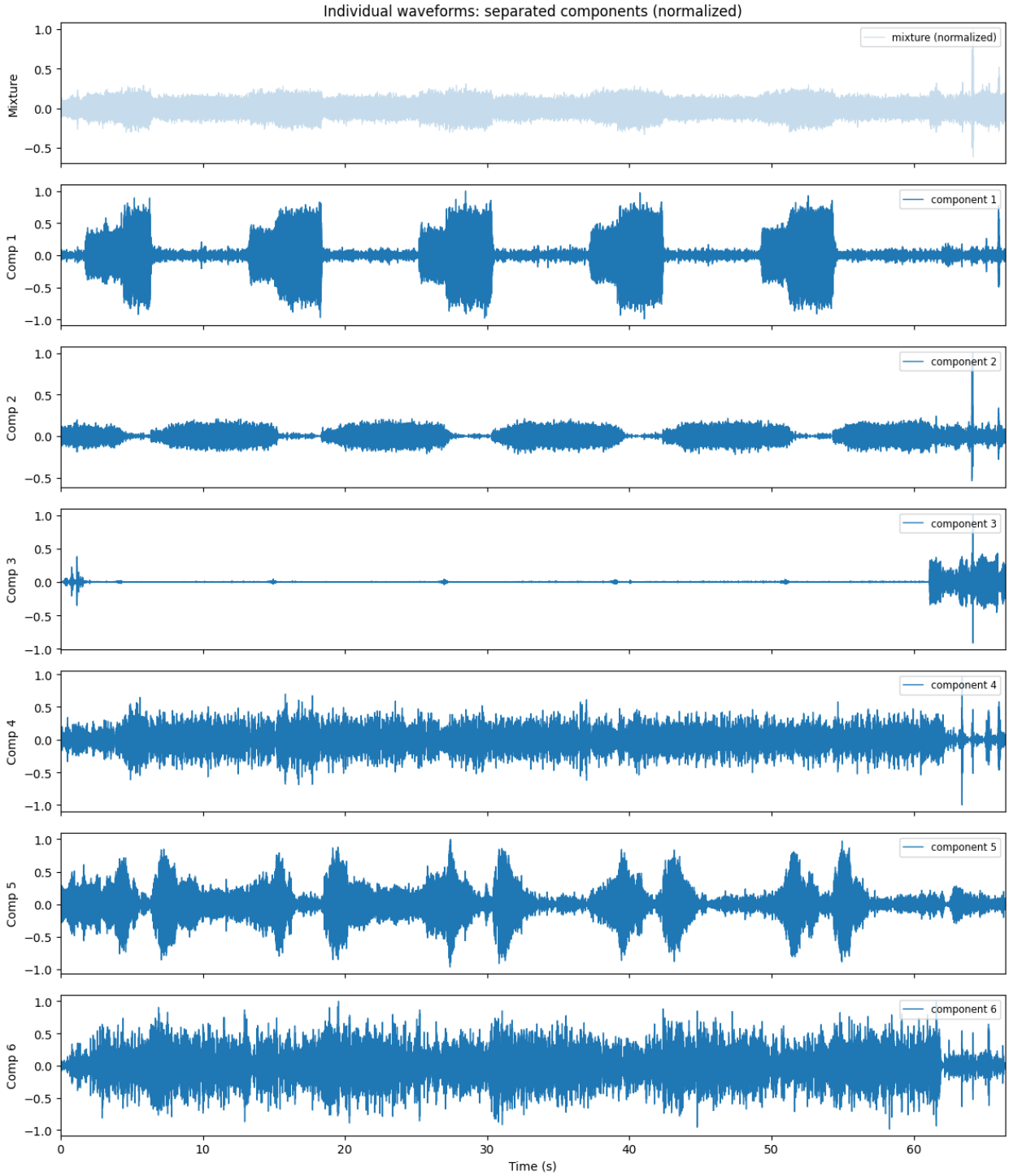





```
Converting 60s-30.m4a to 60s-30.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-30.wav' — duration: 66.45s, sample rate: 48000 Hz,
samples: 3189760
STFT shape: (1025, 6227)
n=2 fit=0.5927
n=3 fit=0.6353
n=4 fit=0.6553
n=5 fit=0.6765
n=6 fit=0.6876
Chosen components: 6
```

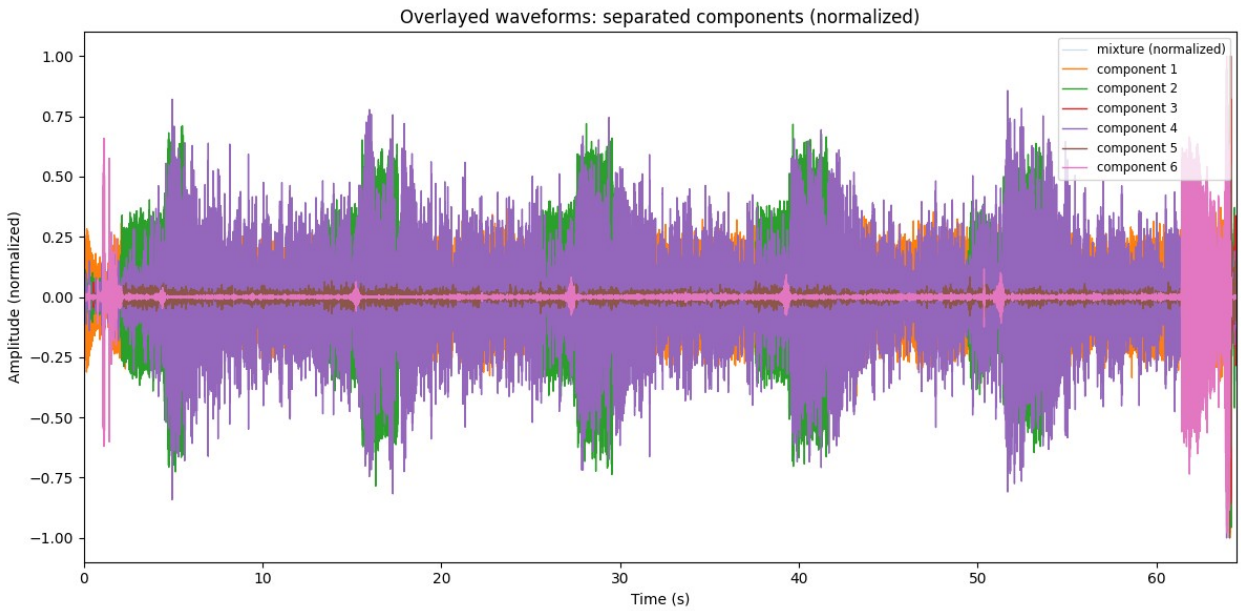
```
Saved outputs: ['60s-30_1.wav', '60s-30_2.wav', '60s-30_3.wav', '60s-30_4.wav', '60s-30_5.wav', '60s-30_6.wav']
```

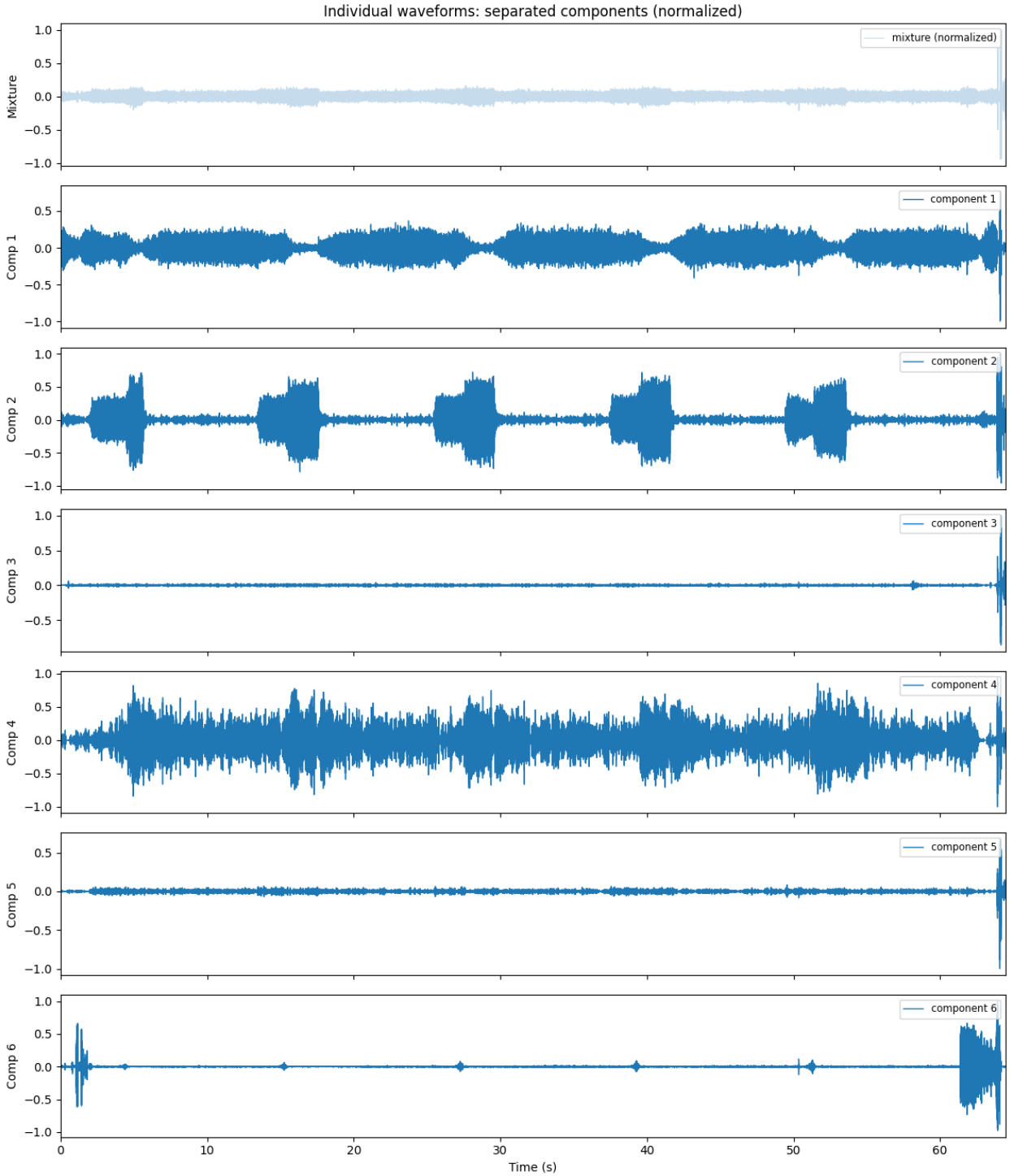





```
Converting 60s-20.m4a to 60s-20.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-20.wav' — duration: 64.47s, sample rate: 48000 Hz,
samples: 3094528
STFT shape: (1025, 6041)
```

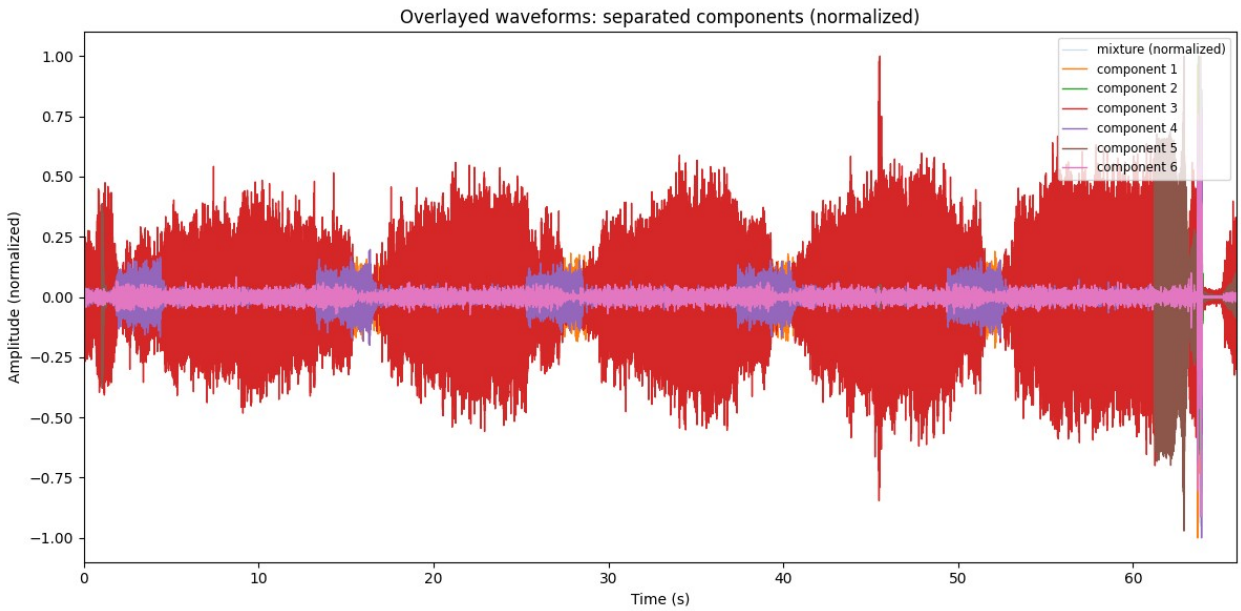
```
n=2 fit=0.5314
n=3 fit=0.5573
n=4 fit=0.6008
n=5 fit=0.6574
n=6 fit=0.6705
Chosen components: 6
Saved outputs: ['60s-20_1.wav', '60s-20_2.wav', '60s-20_3.wav', '60s-20_4.wav', '60s-20_5.wav', '60s-20_6.wav']
```

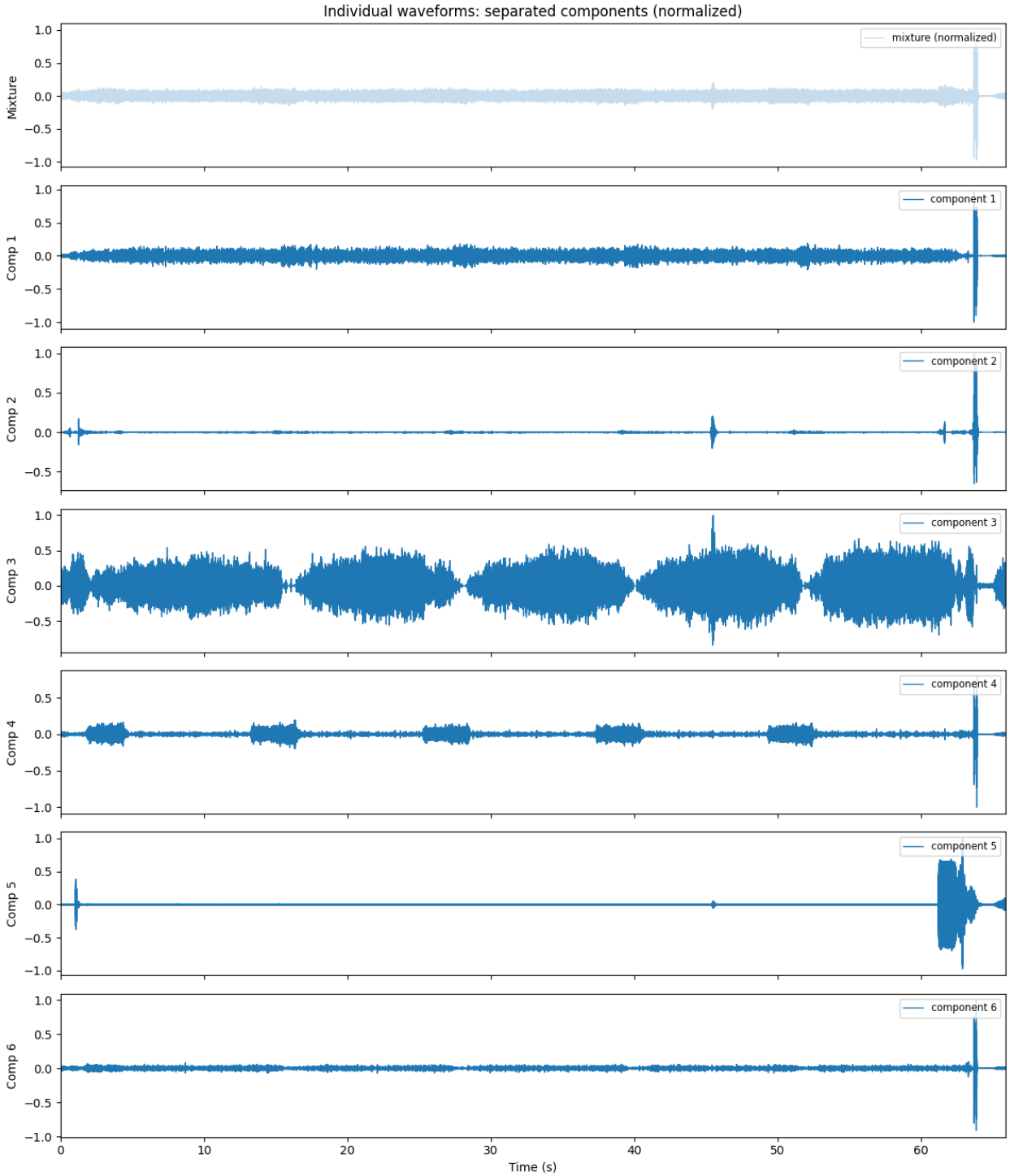





```
Converting 60s-10.m4a to 60s-10.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '60s-10.wav' — duration: 65.92s, sample rate: 48000 Hz,
samples: 3164160
STFT shape: (1025, 6177)
```

```
n=2 fit=0.5059
n=3 fit=0.5390
n=4 fit=0.5865
n=5 fit=0.6201
n=6 fit=0.6419
Chosen components: 6
Saved outputs: ['60s-10_1.wav', '60s-10_2.wav', '60s-10_3.wav', '60s-10_4.wav', '60s-10_5.wav', '60s-10_6.wav']
```






```
Converted .wav files: ['60s-100.wav', '60s-90.wav', '60s-80.wav',
'60s-70.wav', '60s-60.wav', '60s-50-x.wav', '60s-50.wav', '60s-
40.wav', '60s-30.wav', '60s-20.wav', '60s-10.wav']
```